

# Solving Sudoku with Ant Colony Optimization

Huw Lloyd, *Member, IEEE*, and Martyn Amos

**Abstract**—In this paper we present a new algorithm for the well-known and computationally-challenging Sudoku puzzle game. Our Ant Colony Optimization-based method significantly out-performs the state-of-the-art algorithm on the hardest, large instances of Sudoku. We provide evidence that – compared to traditional backtracking methods – our algorithm offers a much more efficient search of the solution space, and demonstrate the utility of a novel anti-stagnation operator. This work lays the foundation for future work on a general-purpose puzzle solver, and establishes Japanese pencil puzzles as a suitable platform for benchmarking a wide range of algorithms.

**Index Terms**—Ant Colony Optimization, Sudoku, Puzzle Games.

## I. INTRODUCTION

Sudoku is a well-known logic-based puzzle game that was first published in 1979 under the name of “Number Place”. It was popularised in Japan in 1984 by the puzzle company Nikoli, and later named “Sudoku”, which roughly translates to “single digits”. The puzzle gained attention in the West in 2004, after *The Times* published its first Sudoku grid at the instigation of Hong Kong-based judge Wayne Gould, who first encountered the puzzle in 1997, and developed a computer program to automatically generate instances. Sudoku is now a global phenomenon, and many newspapers now carry it alongside their existing crosswords (see [1] for a general history of the puzzle).

The simplest variant of Sudoku uses a  $9 \times 9$  grid of cells divided into nine  $3 \times 3$  subgrids (Figure 1 (left)). As we later demonstrate, the problem scales to larger grids, but, for the moment, we focus on the most familiar variant. The aim of the puzzle is to fill the grid with digits such that each row, each column, and each  $3 \times 3$  subgrid contains all of the digits 1-9 (Figure 1 (right)). An instance of Sudoku provides, at the outset, a partially-completed grid, but the difficulty of any grid derives more from the range of techniques required to solve it than the number of cell values that are provided for the player.

Formally, a Sudoku problem of order  $n = 3$  is made up of a grid of *cells* (or squares), arranged into  $3 \times 3$  subgrids known as *boxes*. A *unit* is a row, column or box, each containing exactly nine cells. A problem is solved when *each unit* (that is, every row, column and box) contains a permutation of the digits 1...9 [2]

Any given cell has exactly three units and 20 *peers*; the units are the row, column and box in which the cell resides, and the set of peers is made up of the other cells in those units

H. Lloyd is with the Department of Computing and Mathematics, Manchester Metropolitan University, Manchester, UK (email: huw.lloyd@mmu.ac.uk).

M. Amos is with the Department of Computer and Information Sciences, Northumbria University, UK (email: martyn.amos@northumbria.ac.uk).

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
|   | 6 |   |   |   |   | 5 |   | 2 |
|   | 3 |   |   |   | 7 |   |   |   |
|   | 2 |   | 3 |   | 6 |   |   | 7 |
| 8 | 7 | 3 |   | 2 | 1 | 4 | 5 |   |
| 9 | 4 |   | 5 |   |   |   |   |   |
|   | 1 |   |   |   | 4 |   |   |   |
|   |   |   |   | 9 | 5 |   |   | 4 |
| 3 | 9 | 4 | 8 | 1 |   |   | 7 | 5 |
|   | 5 | 1 |   | 6 | 3 |   | 9 | 8 |

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 4 | 6 | 7 | 1 | 8 | 9 | 5 | 3 | 2 |
| 1 | 3 | 8 | 2 | 5 | 7 | 9 | 4 | 6 |
| 5 | 2 | 9 | 3 | 4 | 6 | 1 | 8 | 7 |
| 8 | 7 | 3 | 6 | 2 | 1 | 4 | 5 | 9 |
| 9 | 4 | 2 | 5 | 3 | 8 | 7 | 6 | 1 |
| 6 | 1 | 5 | 9 | 7 | 4 | 8 | 2 | 3 |
| 2 | 8 | 6 | 7 | 9 | 5 | 3 | 1 | 4 |
| 3 | 9 | 4 | 8 | 1 | 2 | 6 | 7 | 5 |
| 7 | 5 | 1 | 4 | 6 | 3 | 2 | 9 | 8 |

Fig. 1. The structure of a Sudoku puzzle instance (left), and its solution (right).

(that is,  $2 \times 8 = 16$  neighbours in the relevant row and column, plus 4 other cells occupying the same box; see Figure 2).

Sudoku is an NP-complete problem [3], as first shown in [4] via a reduction from the Latin Square Completion problem [5]. As such, the problem offers itself as a useful benchmark challenge, and a number of different types of algorithm have been proposed for its solution (see the next Section for a more detailed discussion of these). However, we also consider the argument that “We should develop AI methods that work with not just one game, but with any game (within a given range) that the method is applied to” [6]. That is, rather than developing a multitude of algorithms to play one specific game, we should seek methods that find broader applicability, across a *range* of games. Although the algorithm we present here is demonstrated in the context of Sudoku, we later show how its lack of reliance on any heuristic information (that is, game-specific “hints”) means that it may be applied to a number of different puzzle games.

While such puzzle games may, superficially, appear to lack “real world” relevance, they in fact offer a significant challenge for general-purpose AI methods; as argued in [6], “We need game playing benchmarks and competitions capable of expressing any kind of game, including puzzle games, 2D arcade games, text adventures, 3D action-adventures and so on; this is the best way to test general AI capacities and reasoning skills.” While our algorithm could not be described as “general purpose”, this does serve to underscore the importance of the puzzle game domain.

The rest of the paper is structured as follows: in Section II we briefly review closely-related recent work on the application of various algorithms to Sudoku. This motivates the description, in Section III of our own method, based on Ant Colony Optimization (ACO), which introduces a novel operator which we call *Best Value Evaporation*. In Section IV we present the results of experimental investigations, which confirm (1) that our algorithm out-performs existing methods, and (2) that BVE is a necessary addition to the basic ACO

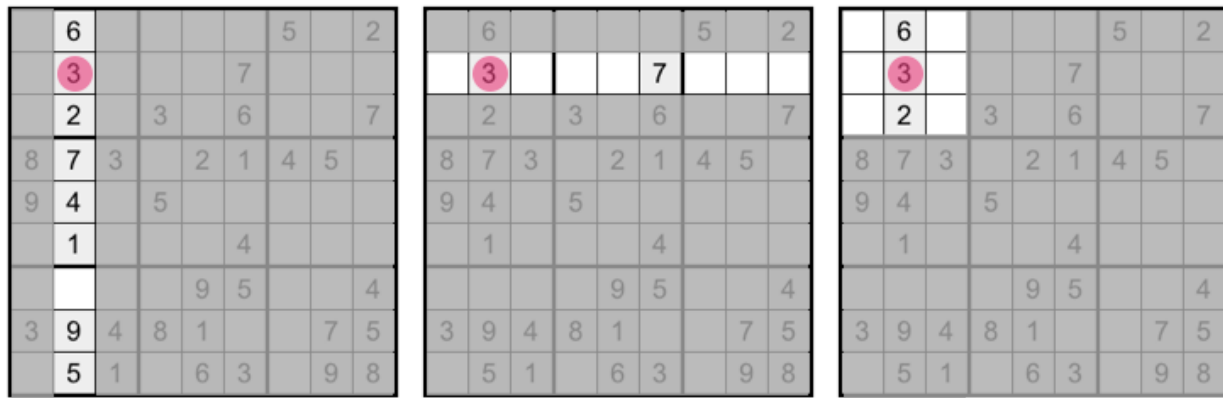


Fig. 2. Units and peers for a specific highlighted cell. The *units* (from right to left, column, row, and block) are highlighted in white. The union of the three units, that is all the white cells, are the *peers*.

algorithm for solving large Sudoku instances. We conclude in Section V with a discussion of our findings, and discuss possible future work in this area.

## II. RELATED WORK

We first consider a “traditional” backtracking approach to solving Sudoku. The *Exact Cover Problem* [7] is a type of constraint satisfaction problem which may be phrased as follows: given a binary matrix, find a *subset* of rows in which each column sums to 1 (that is, find a set of rows in which each column contains only a single 1). In [8], Knuth describes the “dancing links” implementation of his Algorithm X (called DLX), a “brute force” backtracking algorithm for Exact Cover. As any Sudoku puzzle may be transformed into an instance of Exact Cover [9], DLX naturally offers an effective solution method for Sudoku [10].

In [2], Peter Norvig presents an alternative approach, based on *constraint propagation* followed by a search process (we discuss this in more detail shortly). Other notable approaches to solving Sudoku include formal logic [11], an artificial bee colony algorithm [12], constraint programming [13], [14], evolutionary algorithms [15], [16], [17], [18], particle swarm optimisation [19], [20], simulated annealing [21], tabu search [22], and entropy minimization [23]. As this diverse set of solution methods demonstrates, Sudoku offers a challenging yet conceptually simple test-bed for the comparative analysis of algorithms for problems involving complex reasoning.

In this paper, we focus on the application of ACO to the solution of Sudoku. ACO is a population-based search method inspired by the foraging behaviour of ants [24], [25], and it has been successfully applied to a wide range of computational problems (see [26], [27] for overviews of both the algorithm and its applications). The basic ACO algorithm uses a population of “ants” (agents), which individually explore a given problem space and incrementally construct a solution, combined with a global “pheromone” data structure, which is used to inform decisions taken by the ants. Essentially, each ant moves individually on some problem representation (for example, a graph), gradually building a solution and probabilistically choosing its next move according to pheromone concentrations. Components with more pheromone are more

likely to be selected. After a single population iteration, the best solution according to some objective function is selected from the population, and the components it contains (e.g., edges in the graph) are given additional pheromone. In this way, the population rapidly converges on high-quality solutions, although premature or sub-optimal convergence is discouraged through the continuous “evaporation” of pheromone concentrations. Some ACO variants include *local* pheromone operators, which allow individual ants to record information about their traversal during the solution construction process; for example, ants may reduce the global pheromone value associated with components as they are added to a solution, to discourage following ants from taking the same path.

The archetypal ACO algorithm was named “Ant system” [25], and this was applied to the well-known Travelling Salesman Problem as follows: each edge connecting two cities has a pheromone value, and the probability of an edge being selected by an ant is a function of both its pheromone concentration and its distance from the ant’s current location. This process thus combines the autocatalytic power of the global pheromone network with a greedy local search heuristic. Each ant also maintains a “tabu” list of cities that it has visited, and an ant may not re-visit any city on its list. Once it has visited all cities, an ant then deposits an amount of global pheromone which is inversely proportional to the length of its tour; that is, shorter tours deposit more pheromone. Once all ants have completed this process, the global pheromone matrix is evaporated, thus gradually removing the remnants of sub-optimal tours that persist over time. Dorigo *et al.* [25] demonstrate that positive feedback, combined with local search, can offer a heuristic that is robust, versatile, broadly applicable, and amenable to parallelization, because of its inherent population-based structure. Since the publication of the original paper, ACO is now a well-established method [28].

In [29], Mantere presents a hybrid ACO/genetic algorithm approach to Sudoku, which combines global (evolutionary) search with greedy local (ACO-based) search. Schiff [30] and Sabuncu [31] also present relatively recent work on applying ACO to Sudoku, but, in both cases, the performance of the algorithm is relatively poor. Another nature-inspired approach was used by [12], who used a variant of the artificial

bee colony algorithm to solve  $9 \times 9$  Sudoku puzzles. The algorithm was able to solve some difficult instances (such as the *AI Escargot* instance[32]) but the runtime performance is relatively poor with an average solution time of over 6 minutes for difficult instances.

For the purposes of comparison, in this paper we focus mainly on the work of Musliu, *et al.* [33], who present an iterated local search algorithm with constraint programming which represents the state-of-the-art in stochastic search algorithms for the Sudoku problem, plus the algorithms of Knuth [8] and Norvig [2].

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
|   | 6 |   |   |   |   | 5 |   | 2 |
|   | 3 |   |   |   | 7 |   |   |   |
|   | 2 |   | 3 |   | 6 |   |   | 7 |
| 8 | 7 | 3 |   | 2 | 1 | 4 | 5 |   |
| 9 | 4 |   | 5 |   |   |   |   |   |
|   | 1 |   |   |   | 4 |   |   |   |
|   |   |   |   | 9 | 5 |   |   | 4 |
| 3 | 9 | 4 | 8 | 1 |   |   | 7 | 5 |
|   | 5 | 1 |   | 6 | 3 |   | 9 | 8 |

|       |   |     |    |    |   |      |     |     |
|-------|---|-----|----|----|---|------|-----|-----|
| 4     | 6 | 7   | 1  | 8  | 9 | 5    | 3   | 2   |
| 15    | 3 | 589 | 12 | 45 | 7 | 189  | 468 | 169 |
| 15    | 2 | 589 | 13 | 45 | 6 | 189  | 48  | 7   |
| <hr/> |   |     |    |    |   |      |     |     |
| 8     | 7 | 3   | 69 | 2  | 1 | 4    | 5   | 69  |
| 9     | 4 | 26  | 15 | 37 | 8 | 17   | 26  | 136 |
| 256   | 1 | 256 | 69 | 37 | 4 | 1789 | 268 | 369 |
| <hr/> |   |     |    |    |   |      |     |     |
| 26    | 8 | 26  | 17 | 9  | 5 | 13   | 1   | 4   |
| 3     | 9 | 4   | 18 | 1  | 2 | 16   | 7   | 5   |
| 7     | 5 | 1   | 4  | 6  | 3 | 12   | 9   | 8   |

Fig. 3. Instance from Figure 1 (left), and (right) cell value sets after initial pass of constraint propagation algorithm. The value sets are represented as strings of allowable digits for the cell, for example ‘589’ represents the set of values {5, 8, 9}.

### III. OUR ALGORITHM

In [2], Norvig describes a two-component approach to solving Sudoku, using a combination of *constraint propagation* (CP) and *search*. CP ensures that the “rules” of Sudoku are observed, and repeatedly prunes the *value set* of each cell (that is, the set of possible values that cells might take). Importantly, by using CP during search, we effectively “parallelise” the process, by eliminating large numbers of possible cell values every time we fix a cell’s value; selecting a specific value for a cell immediately rules out that value’s presence in a large number of other cells. In [2], Norvig combines CP with a recursive depth-first search which, at each iteration, selects the cell with the smallest value set and then chooses the first numeric ordered value for that cell. This essentially maximises the probability of “guessing correctly”, and is referred to as the *Minimum Remaining Values Heuristic*.

Here, we present a variant of constraint propagation inspired by Norvig’s method, and use ACO (rather than depth-first search) to search the space of solutions. We now describe our CP method in more detail. For clarity, this is written in terms of the  $9 \times 9$  Sudoku puzzle, but the method generalises trivially to larger sizes (e.g.  $16 \times 16$ ,  $25 \times 25$ ).

#### A. Constraint propagation

Throughout the constraint propagation (CP) process, each cell maintains its value set – a list of *possible values* it might take; every cell starts with the same value set,  $[1 \dots 9]$ . Once a set has been reduced to a single value, we call that value *fixed* for that cell. Our CP algorithm implements two basic rules, which are applied to a cell’s peers when it has its value fixed:

- 1) Eliminate from a cell’s value set all values that are *fixed* in any of the cell’s *peers*.
- 2) If any values in a cell’s value set are in the only possible place *in any of the cell’s units*, then fix that value.

Note that since this can lead to other cells having their values fixed, the procedure is recursive, and terminates when no further changes are possible.

In Figure 3 we show the instance from Figure 1 after the *initial* pass of our CP algorithm, which occurs when the board is set up, and before any search is performed. For easy cases, the application of the CP algorithm is often sufficient to solve the board, and no further search is required (see Section IV for a discussion). However, in most cases, some search will be required, and we now describe our ACO-based method for this.

#### B. Our ACO algorithm

Our algorithm is based on Ant Colony System (ACS), which is a variant of ACO introduced in [34]. We first give an informal description of the algorithm, and then formally specify its various components.

At each population-level iteration, every ant works independently on its own copy of the board. However, the global pheromone matrix persists across iterations, allowing for a combination of local search and global positive feedback to occur (i.e., when the best ant in each iteration updates the global pheromone). The ants move round their boards *in parallel*; the ant system iterates over the ants in turn, calling a step function which moves each ant one step. This enables ants to discourage others from following the same path through the local pheromone mechanism. The outer loop of the ant system update therefore iterates  $c$  times, where  $c$  is the number of cells, and at each iteration requests that each ant makes a single step.

As previously stated, once the initial pass of the CP has been completed, then most cells will have a set of possible values. The aim of each ant, in a single population-level iteration, is to fix as many cell values as possible. Each ant starts on a different, randomly-selected cell, and then iterates over all cells on the board. We simply move from one cell to the next because what is important is not the “next cell”, but the *value assigned* to the next cell encountered. Whenever it leaves a cell that does not have a fixed value (that is, a cell with a number of possible values), an ant must make a decision on which element of that cell’s value set to choose, thus setting the cell to that value. Importantly, as soon as an ant sets the value of a cell, the constraints that it introduces are propagated across the board.

Decisions on which value to choose are based on relative pheromone levels, which are assigned to each possible value. These are stored in a *pheromone matrix*, which keeps track of a single pheromone amount for each possible value in each cell. This is, for an order-3 ( $9 \times 9$ ) Sudoku puzzle, a matrix of  $81 \times 9$  values, with each cell corresponding to the pheromone level for each possible value ( $1 \dots 9$ ) in a cell (indexed  $1 \dots 81$ ). Depending on the “greediness” of the selection, either the value with the highest pheromone value is chosen, or a weighted (roulette) selection is made.

---

**Algorithm 1:** Our ACO algorithm for Sudoku

---

```

1 read in puzzle;
2 for all cells with fixed values do
3   propagate constraints (according to Section III-A);
4 end
5 initialize global pheromone matrix;
6 while puzzle is not solved do
7   give each ant a local copy of puzzle;
8   assign each ant to a different cell;
9   for number of cells do
10    for each ant do
11      if current cell value not fixed then
12        choose value from current cell's value set;
13        fix cell value;
14        propagate constraints;
15        update local pheromone;
16      end
17    end
18    move to next cell;
19  end
20  find best ant;
21  do global pheromone update;
22  do best value evaporation;
23 end

```

---

After the cell's value is set, the standard ACS *local* pheromone operator is applied, which reduces the probability of that value being selected by the following ant, thus preventing early convergence.

Once all ants have covered every square of the board, we then perform the *global* pheromone update, which rewards only the best solution found so far (the *global best*, in line with ACS principles). We characterise the “best” solution, at each iteration, as the sequence of value selections that lead to the greatest number of cells having their values fixed; the best solution is effectively the one found by the ant that “guesses” correctly the highest number of times. However, at this point, we introduce a novel variation to the standard ACS algorithm, which we call *best value evaporation* (BVE). In what follows, “best value” refers to an amount of pheromone that is added to the global pheromone matrix whenever the best solution is identified within a generation, and this value is itself subject to evaporation, along with the component pheromone values.

In *standard* ACS, the global pheromone operator increases the pheromone concentrations of all components of the global best solution with an amount of pheromone that is directly proportional to the *absolute quality* of that solution. However, this can gradually lead to *stagnation*, where all ants end up selecting the same route. Instead, the amount of pheromone that is added globally, which we call the *best value*, is measured in terms of the *proportionate quality* of the best solution found so far (Equation 5). Importantly, the best value itself is subject to evaporation over time, which prevents “lock in”; taken together, these two components of BVE prevent premature stagnation, which is confirmed by our later experimental observations.

We give a pseudo-code description of our approach in Algorithm 1, components of which we now formally specify.

**Line 5:** For a Sudoku puzzle of dimension  $d$  we define a two-dimensional global pheromone matrix,  $\tau$ , in which each element is denoted as  $\tau_i^k$ , where  $i$  is the cell index ( $1 \leq i \leq d^2$ ) and  $k$  is a possible value for the cell ( $k \in [1, d]$ ).  $\tau_i^k$  represents the pheromone level associated with value  $k$  in cell  $i$ . Each element of the matrix is initialised to some fixed value,  $\tau_0$  (we use a value of  $1/c$ , where  $c = d^2$  is the total number of cells on the board).

**Line 12:** Where an ant has a choice of a number of values in an “open” cell (i.e., one which does not yet have its value fixed), then we define the *value set*,  $v_i$  of cell  $i$  as the set of all available values for that cell, from which we have to choose one. We have a choice of two methods to use when making a selection; we might make a *greedy* selection, in which case the member of  $v_i$  with the highest pheromone concentration is selected, or we might make a *weighted* (i.e., “roulette wheel”) selection, in which case the selection probabilities are proportional to the pheromone associated with the available choices. The relative probabilities of each type of selection are determined by the *greediness* parameter,  $q_0 \in [0, 1]$ . A value selection,  $s$ , is therefore made according to

$$s = \begin{cases} \operatorname{argmax}_{k \in v_i} \{\tau_i^k\} & \text{if } q < q_0 \\ R & \text{otherwise} \end{cases} \quad (1)$$

where  $q \in [0, 1]$  is a uniform random deviate, and  $R$  is a selection from  $v_i$  made according to the probability distribution

$$p_i^k = \frac{\tau_i^k}{\sum_{j \in v_i} \tau_i^j}, \quad k \in v_i \quad (2)$$

where  $p_i^k$  is the probability of selecting choice  $k$  from  $v_i$ .

If a cell has a value set of size zero (that is, it cannot have its value fixed due to other cells being fixed and the constraints thus introduced), then we mark it as a “fail cell”; the number of fail cells is later subtracted from the number of cells to be fixed when we calculate the quality of a solution (see note below, for Line 20).

**Line 15:** The local pheromone update operator is used to make selected values less attractive in subsequent iterations, thus promoting exploration of the solution space. The local pheromone update is handled as follows; every time an ant selects a value,  $s$ , at cell  $i$ , its pheromone value in the matrix is updated as follows:

$$\tau_i^s \leftarrow (1 - \xi)\tau_i^s + \xi\tau_0 \quad (3)$$

with  $\xi = 0.1$  (the standard setting for ACS).

**Line 20:** In order to perform the global pheromone update, we must first find the best-performing ant. At each iteration, each ant  $n$  of the  $m$  ants keeps track of the number of cells,  $f_n$ ,  $n \in \{1 \dots m\}$ , that it has managed to set to a specific

value. The value of  $f_n$  corresponding to the iteration-best ant is  $f_{\text{best}}$ , given by

$$f_{\text{best}} = \max_{n \in \{1 \dots m\}} f_n. \quad (4)$$

We then calculate the amount of pheromone to add,  $\Delta\tau$ , as follows:

$$\Delta\tau = \frac{c}{c - f_{\text{best}}} \quad (5)$$

where  $c$  is the total number of cells on the board. If the value of  $\Delta\tau$  exceeds the current “best pheromone to add” value,  $\Delta\tau_{\text{best}}$  (a quantity initialized to 0 at the beginning of the run), then we set  $\Delta\tau_{\text{best}} \leftarrow \Delta\tau$ , and replace the current best solution with the solution found by the iteration-best ant.

**Line 20:** We then update all pheromone values corresponding to values in the current best solution, where  $\rho \in [0, 1]$  is the standard evaporation parameter:

$$\tau_i^s \leftarrow (1 - \rho)\tau_i^s + \rho\Delta\tau_{\text{best}}. \quad (6)$$

Note that in ACS, there is no global evaporation of pheromone; the global pheromone update (equation 6) is only applied to pheromone values corresponding to fixed values in the *best* solution; the evaporation parameter  $\rho$  represents the “volatility” of the deposited pheromone, and is used to tune the convergence rate of the algorithm.

**Line 22:** In order to prevent “lock in”, we then additionally apply evaporation to the current best pheromone value,  $\Delta\tau_{\text{best}}$ :

$$\Delta\tau_{\text{best}} \leftarrow \Delta\tau_{\text{best}} \times (1 - \rho_{\text{BVE}}) \quad (7)$$

where  $\rho_{\text{BVE}} \in [0, 1]$  is a parameter which controls the rate of evaporation of the best pheromone value.

#### IV. EXPERIMENTAL RESULTS

Our ant colony algorithm (ACS) was evaluated by comparing it with (1) iterated local search code from Musliu *et al.* (ILS) [33], (2) a C++ implementation of the Dancing Links algorithm (DLX) [35], and (3) our own implementation of backtracking search, using the minimum remaining values heuristic, which uses the same problem representation and constraint propagation code as the ant colony algorithm (BS). The code presented in [33] was itself compared against a number of other stochastic algorithms, and was shown to be the best performing. We include the Dancing Links and backtracking algorithms for comparison with deterministic, exhaustive search. Furthermore, including a backtracking search which uses the same underlying constraint propagation code allows us to evaluate the effectiveness of the ant colony algorithm in searching the problem space, independent of the details of the underlying implementation.

We conducted experiments using a number of logic-solvable  $9 \times 9$  instances from the literature, as well as randomly generated  $9 \times 9$ ,  $16 \times 16$  and  $25 \times 25$  ‘general’ instances (which do not necessarily have a unique solution). In all the experiments, we evaluated the algorithms for success rate over a number of trials or instances, subject to a timeout, and the mean time to solution. This is the same as the evaluation

conducted by [33] of their algorithm against a number of competitors, and gives a measure of the practical applicability of the algorithm in a time-constrained environment. In all cases, we measured the statistical significance of results using non-parametric tests, with a  $p$  value threshold for significance of 0.05. In cases where multiple algorithms are compared together, this significance threshold was modified using the Bonferroni correction. In comparing vectors of solution times, we use the Mann-Whitney U test in cases where the vectors have different lengths, which occurs when the success rates in an experiment differ. This test is appropriate for determining significance of differences in the means of differently-sized samples, when the distribution cannot be assumed to be normal. In cases where all algorithms solved all the instances, we use the Wilcoxon-signed rank test, which tests for significance of difference in the means of paired observations, again with no assumption on the distribution. The success rates are treated as frequencies of a nominal variable (success/fail) for which the Pearson  $\chi^2$  test is appropriate.

#### A. Experimental environment

All of the codes were compiled using the same compiler and optimisation setting (g++ v5.4.0 with -O3). Experiments were run on a machine with an Intel Xeon E5-2460v4 processor with a clock speed of 2.4GHz, running Ubuntu Linux. The parameter settings for the iterated local search solver (ILS) were taken from the recommendations given in [33]. For the ant colony code (ACS), we used the following settings:  $\rho = 0.9$ ,  $q_0 = 0.9$ ,  $\rho_{\text{BVE}} = 0.005$ ,  $m = 10$ . Our code, and all the instance files used for the experiments, may be downloaded from [https://github.com/huwlloyd-mmu/sudoku\\_acs](https://github.com/huwlloyd-mmu/sudoku_acs).

#### B. Logic-solvable $9 \times 9$ instances

We first selected instances based on known difficulty, or on previous use in the literature. We selected the ten instances used in [31] (labelled here *sabuncu1* to *sabuncu10*), five named instances identified by [36] as the most difficult (*Platinum Blond*, *Golden Nugget*, *Red Dwarf*, *coly013*, *tarx0134*), and one instance (*AI Escargot*) [32], commonly regarded as an extremely difficult puzzle. These instances are all *logic solvable*; in other words, they each have a unique solution which can be deduced from the given numbers. We ran the ACS, Iterated Local Search (ILS), Dancing Links (DLX) and backtracking search (BS) algorithms 100 times on each instance, with a timeout of 5 seconds. The puzzles were successfully solved in all cases by all four algorithms; there were no time-outs. Table I shows the timing results for the four algorithms. Since all the instances were solved in all cases, the vectors of times per instance and algorithm are the same length; we therefore use the Wilcoxon Signed Rank test, to determine the significance of differences in the mean time. In all cases we tested the fastest algorithm against the other three, using the Bonferroni correction to lower the significance threshold on the  $p$  value of the tests by a factor equal to the number of tests. We also tested the times obtained by the two stochastic algorithms, ACS and ILS, against each other.

TABLE I

SOLUTION TIMES (MEAN AND STANDARD DEVIATION TIME OVER 100 RUNS) FOR THE LOGIC-SOLVABLE INSTANCES. FIGURES IN BOLD INDICATE TIMES WHICH ARE SIGNIFICANTLY LOWER FOR ONE ALGORITHM COMPARED THE OTHER THREE, BASED ON A WILCOXON SIGNED RANK TEST; THE BONFERONNI CORRECTION IS APPLIED, SO THAT  $p$  VALUES LESS THAN 0.05/3 ARE TAKEN TO BE SIGNIFICANT. ASTERISKS SHOW CASES IN WHICH THE TIMES FOR ILS OR ACS ARE SIGNIFICANTLY LOWER THAN THE OTHER, USING THE WILCOXON SIGNED RANK TEST WITH  $p < 0.05$ .

| Instance             | ACS                               | ILS                    | Solution Time/s                         |                                                     |
|----------------------|-----------------------------------|------------------------|-----------------------------------------|-----------------------------------------------------|
|                      |                                   |                        | DLX                                     | BS                                                  |
| <i>sabuncu1</i>      | $(4.8 \pm 1.84) \times 10^{-5*}$  | $0.00083 \pm 0.00047$  | $0.00105 \pm 0.000362$                  | <b><math>(1.58 \pm 0.651) \times 10^{-6}</math></b> |
| <i>sabuncu2</i>      | $(4.82 \pm 1.73) \times 10^{-5*}$ | $0.00414 \pm 0.00143$  | $0.000937 \pm 0.000308$                 | <b><math>(2.18 \pm 6.45) \times 10^{-6}</math></b>  |
| <i>sabuncu3</i>      | $0.000993 \pm 0.000457*$          | $0.112 \pm 0.0296$     | $0.00104 \pm 0.000366$                  | <b><math>0.000202 \pm 0.0000775</math></b>          |
| <i>sabuncu4</i>      | $0.000625 \pm 0.000708*$          | $0.00859 \pm 0.00229$  | $0.00112 \pm 0.000346$                  | <b><math>0.0001 \pm 0.0000378</math></b>            |
| <i>sabuncu5</i>      | $(4.62 \pm 1.5) \times 10^{-5*}$  | $0.00097 \pm 0.000556$ | $0.00101 \pm 0.000384$                  | <b><math>(1.68 \pm 0.733) \times 10^{-6}</math></b> |
| <i>sabuncu6</i>      | $0.0107 \pm 0.00828*$             | $0.105 \pm 0.027$      | $0.00153 \pm 0.00045$                   | <b><math>0.000775 \pm 0.000273</math></b>           |
| <i>sabuncu7</i>      | $0.00106 \pm 0.000986*$           | $0.0853 \pm 0.0206$    | $0.00102 \pm 0.000318$                  | <b><math>(9.67 \pm 3.75) \times 10^{-5}</math></b>  |
| <i>sabuncu8</i>      | $0.000728 \pm 0.000343*$          | $0.007 \pm 0.00206$    | $0.00107 \pm 0.000374$                  | <b><math>(7.91 \pm 2.74) \times 10^{-5}</math></b>  |
| <i>sabuncu9</i>      | $0.00163 \pm 0.0014*$             | $0.0153 \pm 0.00437$   | $0.00105 \pm 0.000345$                  | <b><math>0.00016 \pm 0.0000579</math></b>           |
| <i>sabuncu10</i>     | $(4.73 \pm 1.85) \times 10^{-5*}$ | $0.00136 \pm 0.000641$ | $0.00104 \pm 0.000363$                  | <b><math>(1.6 \pm 0.693) \times 10^{-6}</math></b>  |
| <i>aiescargot</i>    | $0.0204 \pm 0.0152*$              | $0.152 \pm 0.0328$     | $0.00208 \pm 0.000648$                  | <b><math>0.000475 \pm 0.000182</math></b>           |
| <i>coly013</i>       | $0.0488 \pm 0.0518*$              | $0.702 \pm 0.0685$     | <b><math>0.007 \pm 0.00146</math></b>   | $0.0278 \pm 0.00517$                                |
| <i>goldennugget</i>  | $0.0374 \pm 0.0293*$              | $0.442 \pm 0.0918$     | <b><math>0.00545 \pm 0.00149</math></b> | $0.0152 \pm 0.00304$                                |
| <i>platinumblood</i> | $0.113 \pm 0.0859*$               | $0.131 \pm 0.0223$     | $0.0059 \pm 0.00152$                    | <b><math>0.00268 \pm 0.000923</math></b>            |
| <i>reddwarf</i>      | $0.0404 \pm 0.0354*$              | $0.299 \pm 0.0768$     | <b><math>0.00514 \pm 0.00132</math></b> | $0.00993 \pm 0.00212$                               |
| <i>tarx0134</i>      | $0.0259 \pm 0.0193*$              | $0.851 \pm 0.0699$     | <b><math>0.0185 \pm 0.00303</math></b>  | $0.038 \pm 0.0074$                                  |

The ten puzzles from [31] (*sabuncu1*–*sabuncu10*) are generally solved in less time by all the algorithms than the six harder puzzles. In four cases (*sabuncu1*, *sabuncu2*, *sabuncu5* and *sabuncu10*) the puzzle is solved by a single application of our constraint propagation procedure, so that no searching is required for either the ACS or BS algorithms. The difference in runtimes between the two algorithms for these instances may be explained by the difference in *set-up* times; in the case of ACS, the overhead of creating the ant colony and initializing the pheromone matrix is clearly significant. On these four “trivial” instances, the BS algorithm is the fastest of all (running in times of order a microsecond). DLX requires at least of order a millisecond to solve all the puzzles; in all but the most difficult cases, this time is most likely dominated by the calculations to convert the instance to and from an instance of the exact cover problem.

Overall, we find that the deterministic solvers perform best on these instances. Either DLX or BS is significantly fastest for all of the instances. BS is the best performing overall, and is fastest in twelve of the sixteen instances, with DLX fastest in the other four. ACS is significantly faster than ILS in all cases, and faster than DLX in seven of the sixteen instances.

Finally, we note that the times reported by Sabuncu[31] for their ACO algorithm to solve ten of the instances used here are typically 1 to 3 seconds. This is several orders of magnitude slower than our times using ACS for the same instances which are of the order of milliseconds, or less; this is more than can be accounted for by differences in hardware or efficiency of implementation and although we have not performed a direct comparison with their code, we can safely assume that our algorithm is the better performing of the two.

### C. General instances

Following [14] and [33], we generated random instances for the  $9 \times 9$ ,  $16 \times 16$  and  $25 \times 25$  Sudoku problem. In the latter two cases, subgrids are of size  $4 \times 4$  and  $5 \times 5$  respectively, and

each row, column and subgrid must contain all of the digits  $1 \dots 16$  and  $1 \dots 25$  respectively.

These instances were generated by running the ACS code with an initially blank grid, to produce a set of Sudoku solutions. These are then converted into problem instances by randomly blanking a number of the cells. The instances generated in this way are not guaranteed to have a unique solution. For each of the sizes  $9 \times 9$ ,  $16 \times 16$  and  $25 \times 25$ , we generated 100 instances for fixed cell fractions in steps of 0.05 from 0 to 0.95, giving a total of 6000 individual instances. We ran the ACS, ILS, DLX and BS codes once on each instance, with timeouts set to 5 seconds for the  $9 \times 9$  instances, 20 seconds for  $16 \times 16$  and 120 seconds for  $25 \times 25$ . These timeouts are shorter than those used by [33]; however we ran our experiments on a faster processor, and with compiler optimisations enabled. Taken together, these two differences should amount to a factor of approximately 3 in time. We designed the experiment so that each instance is used for one run; this is preferable to carrying out multiple runs on each of a smaller number of instances [37].

Figures 4, 5 and 6 show the results for average execution time (for successful runs) and success rate for the four algorithms. Summary results are given in Table II and the raw data is given in Table III. In Table III, we indicate in bold quantities which are significantly best of all algorithms, and with asterisks significant differences between the stochastic algorithms ACS and ILS. Statistical significance is tested using the  $\chi^2$  contingency test for the success rates, and the Mann-Whitney U test for the solution times. We use the Mann-Whitney test here as the vectors of times will in general have differing lengths. In cases where we test all algorithms against each other, we apply the Bonferroni correction to modify the  $p$ -value threshold for significance.

As in [33] and [14], we observe a “phase transition” in the difficulty of the instances as a function of the fixed cell fraction; the difficulty is markedly greater at fixed cell fractions of around 40 – 50%. For low values of the fixed cell fraction,



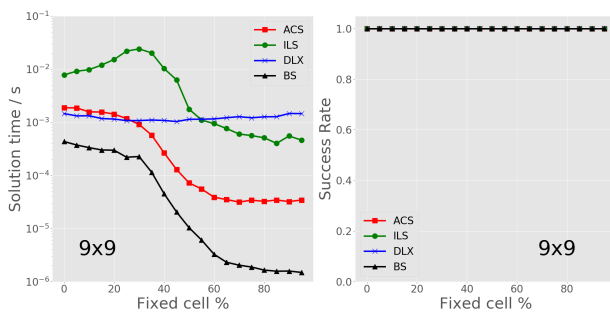


Fig. 4. Plots of solution time (left) and success rate (right) against fixed cell percentage for runs of ACS, ILS, DLX and BS on the  $9 \times 9$  general instances.

the search space is large, but there also exist many possible solutions. As the grid becomes denser, the size of search space decreases as well as the number of possible solutions. At around 45%, the combination of rarity of solutions and the size of the search space leads to a sharp peak in difficulty.

The most difficult puzzles are the  $25 \times 25$  instances with a fixed cell fraction between 40% and 50%. For these fixed cell fractions of 40% and 45%, ACS outperforms the other three algorithms by a *significant* margin; ACS achieves success rates of 98% and 85% (compared to 69% and 10% for ILS, 76% and 49% for DLX, and 21% and 12% for BS). These are the only instances in all the experiments presented for which one algorithm achieved a significantly higher success rate than the other three. The mean times achieved by ACS on these instances are lower than the other three algorithms, but the difference is not statistically significant – this is most likely due to the small samples of times for the three algorithms which recorded low numbers of successes.

It is interesting to note the difference in performance between ACS and BS. These two codes use the same underlying problem representation and constraint propagation code; the only difference between them is the search strategy. This comparison is compelling evidence that ACS is very efficient at searching the solution space, giving markedly improved performance on the hardest instances over an exhaustive search strategy using the same underlying evaluation routines. For the easier instances, BS outperforms ACS, perhaps due to the simplicity of the algorithm which requires very little setup compared to ACS, or transformation to another problem representation, as in DLX.

ACS returns significantly lower runtimes than ILS, the other stochastic search algorithm, in 52 of the 60 instances, whereas ILS is significantly faster than ACS for only two instances. The performance of ACS on these general instances is significantly better than that of ILS both in terms of overall runtime, and success rate on the hardest instances.

#### D. Evaluation of Best Value Evaporation

In order to evaluate the effectiveness of BVE as an anti-stagnation mechanism, we ran experiments using the logic solvable instances (section IV-B) and general instances (section IV-C) using the ACS algorithm with best-value evaporation disabled by setting  $\rho_{BVE} = 0$ . We used all the logic-

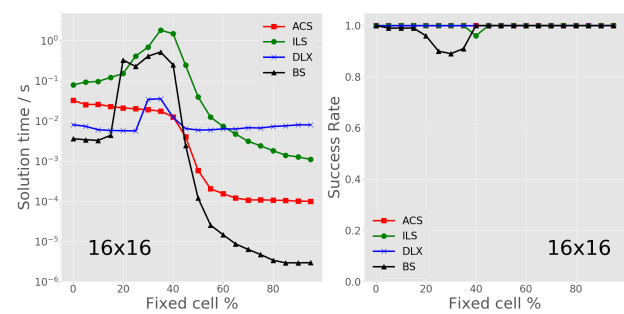


Fig. 5. Plots of solution time (left) and success rate (right) against fixed cell percentage for runs of ACS, ILS, DLX and BS on the  $16 \times 16$  general instances.

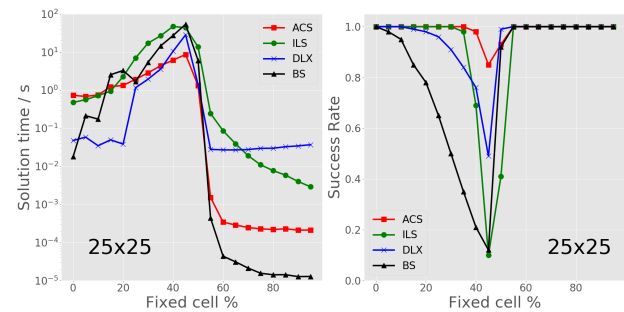


Fig. 6. Plots of solution time (left) and success rate (right) against fixed cell percentage for runs of ACS, ILS, DLX and BS on the  $25 \times 25$  general instances.

solvable instances, and the  $25 \times 25$  general instances (since these are the most challenging). For the named  $9 \times 9$  logic-solvable instances, we find that ACS without BVE performs very poorly on the harder instances (*aiescargot*, *coly013*, *goldennugget*, *platinumblood*, *reddwarf*, *tarx0134*), failing to solve these in most cases (see Table IV). Performance on the ten instances from [31] is similar to BVE, with the exception of *sabuncu6*, with a success rate of 95%. This suggests that these ten instances are not sufficiently difficult to provide a

TABLE II

SUMMARY OF RESULTS ON THE GENERAL INSTANCES (20 FILLED CELL FRACTIONS FOR ORDERS 3, 4 AND 5). THE TABLE SHOWS THE NUMBER OF INSTANCE CATEGORIES FOR WHICH THE ALGORITHM LISTED IN THE FIRST COLUMN (ALGORITHM 0) PRODUCES A SIGNIFICANTLY HIGHER SUCCESS RATE, OR LOWER MEAN SOLUTION TIME, THAN THE OTHER ALGORITHMS (ALGORITHM 1, OR ALL ALGORITHMS). SIGNIFICANCE IS TAKEN AT THE 0.05 LEVEL FOR PAIRWISE COMPARISONS, OR 0.05/3 FOR ONE-AGAINST-ALL COMPARISONS. THIS DATA IS SUMMARIZED FROM TABLE III; DETAILS OF THE STATISTICAL TESTS ARE GIVEN IN SECTION IV.

|         | Algorithm 0 | Algorithm 1 |     |     |    |
|---------|-------------|-------------|-----|-----|----|
|         |             | ACS         | ILS | DLX | BS |
| Success | ACS         | -           | 3   | 3   | 3  |
|         | ILS         | 0           | -   | 0   | 0  |
|         | DLX         | 0           | 0   | -   | 0  |
|         | BS          | 0           | 0   | 0   | -  |
| Time    | ACS         | -           | 52  | 37  | 11 |
|         | ILS         | 3           | -   | 21  | 3  |
|         | DLX         | 21          | 38  | -   | 9  |
|         | BS          | 48          | 52  | 45  | -  |

TABLE III

SOLUTION RATES (SOLVED INSTANCES OUT OF 100) AND TIMES (MEAN AND STANDARD DEVIATION TIME OF SUCCESSFUL RUNS) FOR THE GENERAL INSTANCES.  $O$  IS THE ORDER OF THE PUZZLE (3 FOR  $9 \times 9$ , 4 FOR  $16 \times 16$ , 5 FOR  $25 \times 25$ ) AND  $F$  IS THE PERCENTAGE OF GIVEN CELLS. FIGURES IN BOLD DENOTE QUANTITIES FOR WHICH ONE ALGORITHM IS SIGNIFICANTLY SUPERIOR TO THE OTHER THREE. FOR THE SOLUTION TIMES, THE VECTORS OF TIMES ARE COMPARED USING THE MANN-WHITNEY U TEST. SUCCESS RATES ARE COMPARED USING A  $\chi^2$  CONTINGENCY TEST. IN ALL CASES, THE BONFERONNI CORRECTION IS APPLIED, SO THAT  $p$  VALUES LESS THAN  $0.05/3$  ARE TAKEN TO BE SIGNIFICANT. ASTERISKS SHOW QUANTITIES FOR WHICH EITHER ILS OR ACS ARE SIGNIFICANTLY SUPERIOR TO THE OTHER, USING THE SAME TESTS WITH  $p < 0.05$ .

| O | F(%) | Solution Rate |     |     |     | Solution Time/s                        |                        |                                         |                                                                  |
|---|------|---------------|-----|-----|-----|----------------------------------------|------------------------|-----------------------------------------|------------------------------------------------------------------|
|   |      | ACS           | ILS | DLX | BS  | ACS                                    | ILS                    | DLX                                     | BS                                                               |
| 3 | 0    | 100           | 100 | 100 | 100 | 0.00187 $\pm$ 0.000653*                | 0.00778 $\pm$ 0.0022   | 0.00145 $\pm$ 0.000417                  | <b>0.000433 <math>\pm</math> 0.000152</b>                        |
| 3 | 5    | 100           | 100 | 100 | 100 | 0.00186 $\pm$ 0.000693*                | 0.00912 $\pm$ 0.00302  | 0.00132 $\pm$ 0.000391                  | <b>0.000372 <math>\pm</math> 0.000138</b>                        |
| 3 | 10   | 100           | 100 | 100 | 100 | 0.00156 $\pm$ 0.000563*                | 0.00974 $\pm$ 0.00443  | 0.00132 $\pm$ 0.000509                  | <b>0.000332 <math>\pm</math> 0.000109</b>                        |
| 3 | 15   | 100           | 100 | 100 | 100 | 0.00156 $\pm$ 0.000668*                | 0.0119 $\pm$ 0.00627   | 0.00117 $\pm$ 0.000373                  | <b>0.000300 <math>\pm</math> 0.000101</b>                        |
| 3 | 20   | 100           | 100 | 100 | 100 | 0.00142 $\pm$ 0.000569*                | 0.0152 $\pm$ 0.0101    | 0.00115 $\pm$ 0.00041                   | <b>0.000297 <math>\pm</math> 0.000293</b>                        |
| 3 | 25   | 100           | 100 | 100 | 100 | 0.00117 $\pm$ 0.000368*                | 0.0218 $\pm$ 0.0276    | 0.00107 $\pm$ 0.000338                  | <b>0.000219 <math>\pm</math> 0.000131</b>                        |
| 3 | 30   | 100           | 100 | 100 | 100 | 0.00091 $\pm$ 0.000354*                | 0.024 $\pm$ 0.0266     | 0.00107 $\pm$ 0.000332                  | <b>0.000226 <math>\pm</math> 0.000325</b>                        |
| 3 | 35   | 100           | 100 | 100 | 100 | 0.000572 $\pm$ 0.000256*               | 0.0202 $\pm$ 0.0243    | 0.0011 $\pm$ 0.000341                   | <b>0.000115 <math>\pm</math> 0.0000878</b>                       |
| 3 | 40   | 100           | 100 | 100 | 100 | 0.000266 $\pm$ 0.000207*               | 0.0102 $\pm$ 0.0178    | 0.00108 $\pm$ 0.000355                  | <b>(4.52 <math>\pm</math> 3.7) <math>\times 10^{-5}</math></b>   |
| 3 | 45   | 100           | 100 | 100 | 100 | 0.000129 $\pm$ 0.000111*               | 0.00625 $\pm$ 0.0176   | 0.00103 $\pm$ 0.000349                  | <b>(2.04 <math>\pm</math> 2.57) <math>\times 10^{-5}</math></b>  |
| 3 | 50   | 100           | 100 | 100 | 100 | (7.21 $\pm$ 4.7) $\times 10^{-5}$ *    | 0.00176 $\pm$ 0.000971 | 0.00113 $\pm$ 0.000412                  | <b>(1.03 <math>\pm</math> 1.06) <math>\times 10^{-5}</math></b>  |
| 3 | 55   | 100           | 100 | 100 | 100 | (5.54 $\pm$ 4.98) $\times 10^{-5}$ *   | 0.0011 $\pm$ 0.000671  | 0.00114 $\pm$ 0.000384                  | <b>(6.06 <math>\pm</math> 6.89) <math>\times 10^{-6}</math></b>  |
| 3 | 60   | 100           | 100 | 100 | 100 | (3.87 $\pm$ 1.91) $\times 10^{-5}$ *   | 0.00094 $\pm$ 0.000526 | 0.00115 $\pm$ 0.000382                  | <b>(3.27 <math>\pm</math> 3.68) <math>\times 10^{-6}</math></b>  |
| 3 | 65   | 100           | 100 | 100 | 100 | (3.5 $\pm$ 1.78) $\times 10^{-5}$ *    | 0.00076 $\pm$ 0.000512 | 0.00122 $\pm$ 0.000441                  | <b>(2.31 <math>\pm</math> 2.38) <math>\times 10^{-6}</math></b>  |
| 3 | 70   | 100           | 100 | 100 | 100 | (3.14 $\pm$ 1.34) $\times 10^{-5}$ *   | 0.0006 $\pm$ 0.00051   | 0.00127 $\pm$ 0.000421                  | <b>(2.04 <math>\pm</math> 1.72) <math>\times 10^{-6}</math></b>  |
| 3 | 75   | 100           | 100 | 100 | 100 | (3.41 $\pm$ 1.72) $\times 10^{-5}$     | 0.00056 $\pm$ 0.000571 | 0.00121 $\pm$ 0.000388                  | <b>(1.88 <math>\pm</math> 1.95) <math>\times 10^{-6}</math></b>  |
| 3 | 80   | 100           | 100 | 100 | 100 | (3.24 $\pm$ 1.19) $\times 10^{-5}$     | 0.00051 $\pm$ 0.0005   | 0.00126 $\pm$ 0.000453                  | <b>(1.65 <math>\pm</math> 0.876) <math>\times 10^{-6}</math></b> |
| 3 | 85   | 100           | 100 | 100 | 100 | (3.43 $\pm$ 1.31) $\times 10^{-5}$ *   | 0.0004 $\pm$ 0.00049   | 0.00127 $\pm$ 0.000451                  | <b>(1.57 <math>\pm</math> 0.738) <math>\times 10^{-6}</math></b> |
| 3 | 90   | 100           | 100 | 100 | 100 | (3.21 $\pm$ 1.42) $\times 10^{-5}$     | 0.00055 $\pm$ 0.000497 | 0.00146 $\pm$ 0.00052                   | <b>(1.58 <math>\pm</math> 0.681) <math>\times 10^{-6}</math></b> |
| 3 | 95   | 100           | 100 | 100 | 100 | (3.43 $\pm$ 1.43) $\times 10^{-5}$     | 0.00046 $\pm$ 0.000498 | 0.00145 $\pm$ 0.000511                  | <b>(1.49 <math>\pm</math> 0.700) <math>\times 10^{-6}</math></b> |
| 4 | 0    | 100           | 100 | 100 | 100 | 0.0327 $\pm$ 0.0129*                   | 0.0802 $\pm$ 0.0205    | 0.0081 $\pm$ 0.00213                    | <b>0.00363 <math>\pm</math> 0.00127</b>                          |
| 4 | 5    | 100           | 100 | 100 | 99  | 0.0258 $\pm$ 0.0105*                   | 0.0933 $\pm$ 0.0248    | 0.00739 $\pm$ 0.0042                    | <b>0.00342 <math>\pm</math> 0.000962</b>                         |
| 4 | 10   | 100           | 100 | 100 | 99  | 0.0261 $\pm$ 0.0103*                   | 0.097 $\pm$ 0.0249     | 0.00608 $\pm$ 0.00167                   | <b>0.0033 <math>\pm</math> 0.00186</b>                           |
| 4 | 15   | 100           | 100 | 100 | 99  | 0.0231 $\pm$ 0.00846*                  | 0.122 $\pm$ 0.0411     | 0.00583 $\pm$ 0.00158                   | <b>0.00441 <math>\pm</math> 0.00742</b>                          |
| 4 | 20   | 100           | 100 | 100 | 96  | 0.0212 $\pm$ 0.00958*                  | 0.153 $\pm$ 0.0618     | <b>0.00575 <math>\pm</math> 0.0012</b>  | 0.331 $\pm$ 1.57                                                 |
| 4 | 25   | 100           | 100 | 100 | 90  | 0.0202 $\pm$ 0.00885*                  | 0.414 $\pm$ 0.945      | <b>0.00567 <math>\pm</math> 0.00178</b> | 0.229 $\pm$ 1.87                                                 |
| 4 | 30   | 100           | 100 | 100 | 89  | <b>0.0191 <math>\pm</math> 0.0111*</b> | 0.698 $\pm$ 1.3        | 0.0346 $\pm$ 0.287                      | 0.414 $\pm$ 2.38                                                 |
| 4 | 35   | 100           | 100 | 100 | 91  | <b>0.0176 <math>\pm</math> 0.0102*</b> | 1.84 $\pm$ 2.47        | 0.036 $\pm$ 0.254                       | 0.525 $\pm$ 2.41                                                 |
| 4 | 40   | 100           | 96  | 100 | 100 | 0.0127 $\pm$ 0.00945*                  | 1.5 $\pm$ 1.98         | 0.0123 $\pm$ 0.0363                     | 0.25 $\pm$ 1.27                                                  |
| 4 | 45   | 100           | 100 | 100 | 100 | 0.00406 $\pm$ 0.00392*                 | 0.246 $\pm$ 0.21       | 0.00652 $\pm$ 0.00165                   | <b>0.00248 <math>\pm</math> 0.0076</b>                           |
| 4 | 50   | 100           | 100 | 100 | 100 | 0.000588 $\pm$ 0.000564*               | 0.04 $\pm$ 0.0422      | 0.00596 $\pm$ 0.00192                   | <b>0.000121 <math>\pm</math> 0.000194</b>                        |
| 4 | 55   | 100           | 100 | 100 | 100 | 0.000205 $\pm$ 0.000162*               | 0.0125 $\pm$ 0.00548   | 0.00604 $\pm$ 0.00158                   | <b>(2.55 <math>\pm</math> 2.57) <math>\times 10^{-5}</math></b>  |
| 4 | 60   | 100           | 100 | 100 | 100 | 0.000156 $\pm$ 0.000106*               | 0.00736 $\pm$ 0.00314  | 0.00638 $\pm$ 0.00196                   | <b>(1.47 <math>\pm</math> 1.58) <math>\times 10^{-5}</math></b>  |
| 4 | 65   | 100           | 100 | 100 | 100 | 0.000121 $\pm$ 0.000059*               | 0.00476 $\pm$ 0.00224  | 0.00634 $\pm$ 0.00159                   | <b>(8.72 <math>\pm</math> 9.37) <math>\times 10^{-6}</math></b>  |
| 4 | 70   | 100           | 100 | 100 | 100 | 0.000108 $\pm$ 0.000050*               | 0.00314 $\pm$ 0.00144  | 0.00684 $\pm$ 0.00171                   | <b>(6.35 <math>\pm</math> 6.89) <math>\times 10^{-6}</math></b>  |
| 4 | 75   | 100           | 100 | 100 | 100 | 0.000109 $\pm$ 0.000050*               | 0.00241 $\pm$ 0.00102  | 0.0067 $\pm$ 0.00151                    | <b>(4.73 <math>\pm</math> 5.52) <math>\times 10^{-6}</math></b>  |
| 4 | 80   | 100           | 100 | 100 | 100 | 0.000107 $\pm$ 0.000041*               | 0.00183 $\pm$ 0.000749 | 0.00734 $\pm$ 0.00162                   | <b>(3.41 <math>\pm</math> 2.79) <math>\times 10^{-6}</math></b>  |
| 4 | 85   | 100           | 100 | 100 | 100 | 0.000105 $\pm$ 0.000044*               | 0.00141 $\pm$ 0.000634 | 0.0076 $\pm$ 0.0018                     | <b>(2.94 <math>\pm</math> 1.49) <math>\times 10^{-6}</math></b>  |
| 4 | 90   | 100           | 100 | 100 | 100 | 0.000101 $\pm$ 0.000041*               | 0.00128 $\pm$ 0.000618 | 0.00805 $\pm$ 0.00208                   | <b>(2.94 <math>\pm</math> 1.13) <math>\times 10^{-6}</math></b>  |
| 4 | 95   | 100           | 100 | 100 | 100 | 0.000101 $\pm$ 0.000033*               | 0.00111 $\pm$ 0.000488 | 0.00805 $\pm$ 0.0023                    | <b>(2.96 <math>\pm</math> 1.55) <math>\times 10^{-6}</math></b>  |
| 5 | 0    | 100           | 100 | 100 | 100 | 0.731 $\pm$ 0.724                      | 0.474 $\pm$ 0.0598     | 0.0476 $\pm$ 0.00759                    | <b>0.0178 <math>\pm</math> 0.00351</b>                           |
| 5 | 5    | 100           | 100 | 100 | 98  | 0.682 $\pm$ 0.661                      | 0.561 $\pm$ 0.114*     | 0.0582 $\pm$ 0.173                      | 0.213 $\pm$ 0.672                                                |
| 5 | 10   | 100           | 100 | 100 | 95  | 0.749 $\pm$ 0.931                      | 0.715 $\pm$ 0.185*     | 0.0347 $\pm$ 0.0392                     | 0.173 $\pm$ 0.791                                                |
| 5 | 15   | 100           | 100 | 99  | 85  | 1.23 $\pm$ 1.41                        | 0.943 $\pm$ 0.671      | 0.0494 $\pm$ 0.17                       | 2.54 $\pm$ 12.1                                                  |
| 5 | 20   | 100           | 100 | 98  | 78  | 1.33 $\pm$ 1.53*                       | 2.27 $\pm$ 2.68        | 0.0382 $\pm$ 0.0281                     | 3.27 $\pm$ 13.6                                                  |
| 5 | 25   | 100           | 100 | 96  | 65  | 1.93 $\pm$ 1.69*                       | 7.0 $\pm$ 6.54         | <b>1.16 <math>\pm</math> 5.89</b>       | 1.69 $\pm$ 7.03                                                  |
| 5 | 30   | 100           | 100 | 91  | 50  | 2.85 $\pm$ 2.52*                       | 17.2 $\pm$ 8.8         | <b>1.94 <math>\pm</math> 11.1</b>       | 5.39 $\pm$ 16.5                                                  |
| 5 | 35   | 100           | 98  | 84  | 35  | 4.36 $\pm$ 3.4*                        | 26.7 $\pm$ 10.5        | <b>3.57 <math>\pm</math> 11.0</b>       | 14.5 $\pm$ 29.3                                                  |
| 5 | 40   | <b>98*</b>    | 69  | 76  | 21  | 6.15 $\pm$ 5.61*                       | 47.1 $\pm$ 25.7        | 10.5 $\pm$ 19.5                         | 27.1 $\pm$ 37.3                                                  |
| 5 | 45   | <b>85*</b>    | 10  | 49  | 12  | 8.61 $\pm$ 10.1*                       | 43.4 $\pm$ 26.8        | 28.3 $\pm$ 37.4                         | 53.8 $\pm$ 42.1                                                  |
| 5 | 50   | 93*           | 41  | 99  | 92  | 1.3 $\pm$ 4.82*                        | 13.6 $\pm$ 21.9        | 1.41 $\pm$ 5.73                         | 6.01 $\pm$ 14.9                                                  |
| 5 | 55   | 100           | 100 | 100 | 100 | 0.00152 $\pm$ 0.0082*                  | 0.243 $\pm$ 0.284      | 0.0278 $\pm$ 0.00469                    | <b>0.000435 <math>\pm</math> 0.00288</b>                         |
| 5 | 60   | 100           | 100 | 100 | 100 | 0.000341 $\pm$ 0.000153*               | 0.0857 $\pm$ 0.0275    | 0.027 $\pm$ 0.00411                     | <b>(4.36 <math>\pm</math> 3.19) <math>\times 10^{-5}</math></b>  |
| 5 | 65   | 100           | 100 | 100 | 100 | 0.000287 $\pm$ 0.000105*               | 0.039 $\pm$ 0.019      | 0.0269 $\pm$ 0.00405                    | <b>(3.09 <math>\pm</math> 2.19) <math>\times 10^{-5}</math></b>  |
| 5 | 70   | 100           | 100 | 100 | 100 | 0.000246 $\pm$ 0.000080*               | 0.019 $\pm$ 0.00721    | 0.0276 $\pm$ 0.00416                    | <b>(2.12 <math>\pm</math> 1.12) <math>\times 10^{-5}</math></b>  |
| 5 | 75   | 100           | 100 | 100 | 100 | 0.000227 $\pm$ 0.000063*               | 0.0109 $\pm$ 0.00381   | 0.0295 $\pm$ 0.00426                    | <b>(1.55 <math>\pm</math> 0.596) <math>\times 10^{-5}</math></b> |
| 5 | 80   | 100           | 100 | 100 | 100 | 0.000219 $\pm$ 0.000050*               | 0.0077 $\pm$ 0.00257   | 0.0298 $\pm$ 0.00407                    | <b>(1.41 <math>\pm</math> 0.576) <math>\times 10^{-5}</math></b> |
| 5 | 85   | 100           | 100 | 100 | 100 | 0.000228 $\pm$ 0.000061*               | 0.00584 $\pm$ 0.00194  | 0.0326 $\pm$ 0.00502                    | <b>(1.42 <math>\pm</math> 0.494) <math>\times 10^{-5}</math></b> |
| 5 | 90   | 100           | 100 | 100 | 100 | 0.00021 $\pm$ 0.000047*                | 0.00399 $\pm$ 0.00121  | 0.0343 $\pm$ 0.00527                    | <b>(1.27 <math>\pm</math> 0.232) <math>\times 10^{-5}</math></b> |
| 5 | 95   | 100           | 100 | 100 | 100 | 0.000212 $\pm$ 0.000051*               | 0.00289 $\pm$ 0.00103  | 0.0369 $\pm$ 0.00548                    | <b>(1.28 <math>\pm</math> 0.309) <math>\times 10^{-5}</math></b> |



TABLE IV

PERFORMANCE OF ACS WITH AND WITHOUT BVE ON THE SIXTEEN  $9 \times 9$  LOGIC-SOLVABLE INSTANCES AND GENERAL  $25 \times 25$  INSTANCES. SUCCESS% IS THE NUMBER OF SUCCESSFUL SOLUTIONS FOUND IN 100 RUNS. TIMES ARE GIVEN IN SECONDS, WITH MEAN AND STANDARD DEVIATION OVER 100 RUNS. NUMBERS IN BOLD INDICATE STATISTICALLY SIGNIFICANT DIFFERENCES BETWEEN THE ALGORITHMS, DETERMINED USING THE MANN-WHITNEY U TEST FOR THE TIMES, AND  $\chi^2$  CONTINGENCY TEST FOR THE SUCCESS RATES.

| Instance             | Success %  |        | Solution Time/s                           |                                     |
|----------------------|------------|--------|-------------------------------------------|-------------------------------------|
|                      | BVE        | No BVE | BVE                                       | No BVE                              |
| <i>sabuncu1</i>      | 100        | 100    | $(4.8e \pm 1.84) \times 10^{-5}$          | $(4.55 \pm 1.85) \times 10^{-5}$    |
| <i>sabuncu2</i>      | 100        | 100    | $(4.82 \pm 1.73) \times 10^{-5}$          | $(4.87 \pm 1.82) \times 10^{-5}$    |
| <i>sabuncu3</i>      | 100        | 100    | $0.000993 \pm 0.000457$                   | $0.000993 \pm 0.000454$             |
| <i>sabuncu4</i>      | 100        | 100    | $0.000625 \pm 0.000708$                   | $0.000563 \pm 0.000277$             |
| <i>sabuncu5</i>      | 100        | 100    | $(4.62 \pm 1.5) \times 10^{-5}$           | $(4.8 \pm 1.7) \times 10^{-5}$      |
| <i>sabuncu6</i>      | <b>100</b> | 93     | <b><math>0.0107 \pm 0.00828</math></b>    | $0.561 \pm 0.976$                   |
| <i>sabuncu7</i>      | 100        | 100    | $0.00106 \pm 0.000986$                    | $0.00126 \pm 0.0021$                |
| <i>sabuncu8</i>      | 100        | 100    | $0.000728 \pm 0.000343$                   | $0.000713 \pm 0.000334$             |
| <i>sabuncu9</i>      | 100        | 100    | $0.00163 \pm 0.0014$                      | $0.00225 \pm 0.00324$               |
| <i>sabuncu10</i>     | 100        | 100    | $(4.73 \pm 1.85) \times 10^{-5}$          | $(4.77 \pm 1.84) \times 10^{-5}$    |
| <i>aiescargot</i>    | <b>100</b> | 59     | <b><math>0.0204 \pm 0.0152</math></b>     | $0.949 \pm 1.36$                    |
| <i>coly013</i>       | <b>100</b> | 24     | $0.0488 \pm 0.0518$                       | $0.633 \pm 1.14$                    |
| <i>goldennugget</i>  | <b>100</b> | 34     | <b><math>0.0374 \pm 0.0293</math></b>     | $0.92 \pm 1.15$                     |
| <i>platinumblood</i> | <b>100</b> | 7      | <b><math>0.113 \pm 0.0859</math></b>      | $1.05 \pm 0.905$                    |
| <i>reddwarf</i>      | <b>100</b> | 35     | <b><math>0.0404 \pm 0.0354</math></b>     | $1.64 \pm 1.61$                     |
| <i>tarx0134</i>      | <b>100</b> | 47     | <b><math>0.0259 \pm 0.0193</math></b>     | $1.5 \pm 1.54$                      |
| $25 \times 25 0\%$   | 100        | 100    | $0.731 \pm 0.724$                         | $0.503 \pm 0.347$                   |
| $25 \times 25 5\%$   | 100        | 100    | $0.682 \pm 0.661$                         | $0.469 \pm 0.297$                   |
| $25 \times 25 10\%$  | 100        | 100    | $0.749 \pm 0.931$                         | $0.521 \pm 0.363$                   |
| $25 \times 25 15\%$  | 100        | 100    | $1.23 \pm 1.41$                           | <b><math>0.581 \pm 0.535</math></b> |
| $25 \times 25 20\%$  | 100        | 100    | $1.33 \pm 1.53$                           | $0.792 \pm 0.594$                   |
| $25 \times 25 25\%$  | 100        | 100    | $1.93 \pm 1.69$                           | <b><math>1.08 \pm 0.783</math></b>  |
| $25 \times 25 30\%$  | 100        | 100    | $2.85 \pm 2.52$                           | <b><math>1.77 \pm 1.3</math></b>    |
| $25 \times 25 35\%$  | 100        | 100    | <b><math>4.36 \pm 3.4</math></b>          | $4.59 \pm 8.25$                     |
| $25 \times 25 40\%$  | <b>98</b>  | 83     | $6.15 \pm 5.61$                           | $8.77 \pm 15.6$                     |
| $25 \times 25 45\%$  | <b>85</b>  | 49     | $8.61 \pm 10.1$                           | $9.14 \pm 18.2$                     |
| $25 \times 25 50\%$  | <b>93</b>  | 80     | $1.3 \pm 4.82$                            | $1.52 \pm 11.0$                     |
| $25 \times 25 55\%$  | 100        | 100    | $0.00152 \pm 0.0082$                      | $0.00101 \pm 0.00273$               |
| $25 \times 25 60\%$  | 100        | 100    | <b><math>0.000341 \pm 0.000153</math></b> | $0.000411 \pm 0.000208$             |
| $25 \times 25 65\%$  | 100        | 100    | <b><math>0.000287 \pm 0.000105</math></b> | $0.000321 \pm 0.000105$             |
| $25 \times 25 70\%$  | 100        | 100    | <b><math>0.000246 \pm 0.000081</math></b> | $0.000274 \pm 0.000075$             |
| $25 \times 25 75\%$  | 100        | 100    | <b><math>0.000227 \pm 0.000063</math></b> | $0.000253 \pm 0.000063$             |
| $25 \times 25 80\%$  | 100        | 100    | <b><math>0.000219 \pm 0.000050</math></b> | $0.000243 \pm 0.000055$             |
| $25 \times 25 85\%$  | 100        | 100    | <b><math>0.000228 \pm 0.000061</math></b> | $0.000246 \pm 0.000057$             |
| $25 \times 25 90\%$  | 100        | 100    | <b><math>0.00021 \pm 0.000048</math></b>  | $0.000239 \pm 0.000049$             |
| $25 \times 25 95\%$  | 100        | 100    | <b><math>0.000212 \pm 0.000051</math></b> | $0.000251 \pm 0.000054$             |

good benchmark for solution algorithms: the search space after applying constraints is either too small or, as is the case for four of the instances, non-existent.

We also evaluated BVE using the general  $25 \times 25$  instances. We see that the performance of ACS is *significantly degraded* without the BVE operator. Performance with respect to solution time is degraded to some extent, with significantly shorter times without BVE in three fixed cell fractions, compared to nine which are faster with BVE. The number of failures is significantly higher; for the 45% fixed cell instances for example, the success rate is 58%, compared to 92% with BVE enabled. The average solution time for these instances is 9.1s, well within the timeout of 120s, suggesting that the failures are due to the search stagnating at a local minimum.

## V. CONCLUSIONS

In this paper we presented a new algorithm for the Sudoku puzzle, based on Ant Colony Optimization. Our method includes a new operator, which we call *Best Value Evaporation*, and we show that this addition to the base algorithm is essential for the prevention of premature convergence or stagnation

of solutions. Experiments show that our new algorithm significantly out-performs existing algorithms on the hardest, large instances of Sudoku, and we provide evidence that our method provides a much more efficient search of the solution space than traditional backtracking algorithms for these problems. For smaller or easier instances, we find that direct search algorithms such as Dancing Links or Backtracking Search outperform stochastic algorithms, but these deterministic algorithms perform poorly on the hardest instances. Finally, we find that our algorithm outperforms the state of the art Iterated Local Search algorithm [33] both in terms of runtime and success rates on hard instances.

The growing body of work on the automated solution of pencil puzzles such as Sudoku and Nurikabe suggests that they offer a ready-made algorithmic test-bed. As such, they may provide an additional challenge for general-purpose algorithms (whether AI-based or not), and offer new insights into the solution of constraint satisfaction problems (by, for example, suggesting new ways in which to search the solution space).

Importantly, solvers such as ours can out-perform state-of-the-art methods without any requirement for problem-specific heuristics, which immediately offers two possibilities for fu-

ture work in this area. The first is a “problem agnostic” *general* Japanese pencil puzzle solver, which can solve large instances of any problem in this class. By constructing this solver in a modular fashion, we should easily be able to incorporate any suitable pencil puzzle, which will minimize the amount of effort required in future research. Importantly, this will allow for the rapid (and experimentally consistent) solution of a wide range of pencil puzzles, which will (a) yield good solutions to these problems *per se*, (b) allow for easy comparison of the *properties* of those problems, and (c) provide a ready-made platform for the subsequent investigation of problem-specific heuristics.

## REFERENCES

- [1] J.-P. Delahaye, “The science behind Sudoku,” *Scientific American*, vol. 294, no. 6, pp. 80–87, 2006.
- [2] P. Norvig, “Solving every Sudoku puzzle,” available at <http://norvig.com/sudoku.html>, accessed March 13, 2018.
- [3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. WH Freeman: New York, 1979.
- [4] T. Yato and T. Seta, “Complexity and completeness of finding another solution and its application to puzzles,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 86, no. 5, pp. 1052–1060, 2003.
- [5] C. J. Colbourn, “The complexity of completing partial Latin squares,” *Discrete Applied Mathematics*, vol. 8, no. 1, pp. 25–30, 1984.
- [6] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018.
- [7] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*. Springer, 1972, pp. 85–103.
- [8] D. E. Knuth, “Dancing links,” *arXiv preprint cs/0011047*, 2000.
- [9] M. Hunt, C. Pong, and G. Tucker, “Difficulty-driven Sudoku puzzle generation,” *UMAP Journal*, vol. 29, no. 3, pp. 343–361, 2007.
- [10] S. Fletcher, F. Johnson, and D. R. Morrison, “Taking the mystery out of Sudoku difficulty: an Oracular model,” *UMAP Journal*, vol. 29, no. 3, pp. 327–341, 2007.
- [11] T. Weber, “A SAT-based Sudoku solver,” in *The 12th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR): Short Paper Proceedings*, G. Sutcliffe and A. Voronkov, Eds., 2005, pp. 11–15.
- [12] J. A. Pacurib, G. M. M. Seno, and J. P. T. Yusiong, “Solving Sudoku puzzles using improved artificial bee colony algorithm,” in *Fourth International Conference on Innovative Computing, Information and Control (ICICIC)*. IEEE, 2009, pp. 885–888.
- [13] B. Crawford, M. Aranda, C. Castro, and E. Monfroy, “Using constraint programming to solve Sudoku puzzles,” in *Third International Conference on Convergence and Hybrid Information Technology (ICCHIT)*, vol. 2. IEEE, 2008, pp. 926–931.
- [14] R. Lewis, “Metaheuristics can solve Sudoku puzzles,” *Journal of Heuristics*, vol. 13, no. 4, pp. 387–401, 2007.
- [15] X. Q. Deng and Y. Da Li, “A novel hybrid genetic algorithm for solving Sudoku puzzles,” *Optimization Letters*, vol. 7, no. 2, pp. 241–257, 2013.
- [16] T. Mantere and J. Koljonen, “Solving, rating and generating Sudoku puzzles with GA,” in *IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2007, pp. 1382–1389.
- [17] C. Segura, S. I. V. Peña, S. B. Rionda, and A. H. Aguirre, “The importance of diversity in the application of evolutionary algorithms to the Sudoku problem,” in *IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2016, pp. 919–926.
- [18] Z. Wang, T. Yasuda, and K. Ohkura, “An evolutionary approach to Sudoku puzzles with filtered mutations,” in *IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2015, pp. 1732–1737.
- [19] J. M. Hereford and H. Gerlach, “Integer-valued particle swarm optimization applied to Sudoku puzzles,” in *IEEE Swarm Intelligence Symposium (SIS)*. IEEE, 2008, pp. 1–7.
- [20] A. Moraglio and J. Togelius, “Geometric particle swarm optimization for the Sudoku puzzle,” in *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO)*. ACM, 2007, pp. 118–125.
- [21] Z. Karimi-Dehkordi, K. Zamanifar, A. Baraani-Dastjerdi, and N. Ghasem-Aghaee, “Sudoku using parallel simulated annealing,” in *International Conference in Swarm Intelligence (ICSI)*. Springer, 2010, pp. 461–467.
- [22] R. Soto, B. Crawford, C. Galleguillos, E. Monfroy, and F. Paredes, “A hybrid ac3-tabu search algorithm for solving Sudoku puzzles,” *Expert Systems with Applications*, vol. 40, no. 15, pp. 5817–5821, 2013.
- [23] J. Gunther and T. Moon, “Entropy minimization for solving Sudoku,” *IEEE Transactions on Signal Processing*, vol. 60, no. 1, pp. 508–513, 2012.
- [24] M. Dorigo and G. Di Caro, “Ant colony optimization: a new metaheuristic,” in *Proceedings of the 1999 Congress on Evolutionary Computation (CEC)*, vol. 2. IEEE, 1999, pp. 1470–1477.
- [25] M. Dorigo, V. Maniezzo, and A. Colomi, “Ant system: optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996.
- [26] M. Dorigo and M. Birattari, “Ant colony optimization,” in *Encyclopedia of Machine Learning*. Springer, 2011, pp. 36–39.
- [27] M. López-Ibáñez, T. Stützle, and M. Dorigo, “Ant colony optimization: A component-wise overview,” *Handbook of Heuristics*, pp. 1–37, 2016.
- [28] M. Dorigo and T. Stützle, “Ant colony optimization: overview and recent advances,” in *Handbook of Metaheuristics*. Springer, 2019, pp. 311–351.
- [29] T. Mantere, “Improved ant colony genetic algorithm hybrid for Sudoku solving,” in *Third World Congress on Information and Communication Technologies (WICT)*. IEEE, 2013, pp. 274–279.
- [30] K. Schiff, “An ant algorithm for the Sudoku problem,” *Journal of Automation, Mobile Robotics and Intelligent Systems*, vol. 9, 2015.
- [31] I. Sabuncu, “Work-in-progress: solving Sudoku puzzles using hybrid ant colony optimization algorithm,” in *1st International Conference on Industrial Networks and Intelligent Systems (INISCom)*. IEEE, 2015, pp. 181–184.
- [32] A. Inkala, *AI Escargot - The Most Difficult Sudoku Puzzle*. Lulu.com, Finland, 2007.
- [33] N. Musliu and F. Winter, “A hybrid approach for the Sudoku problem: using constraint programming in iterated local search,” *IEEE Intelligent Systems*, vol. 32, no. 2, pp. 52–62, 2017.
- [34] M. Dorigo and L. M. Gambardella, “Ant colony system: a cooperative learning approach to the Traveling Salesman Problem,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [35] J. Laire, “dlx-cpp,” available at <https://github.com/jlaire/dlx-cpp>, accessed April 23, 2018.
- [36] M. Ercsey-Ravasz and Z. Toroczkai, “The chaos within Sudoku,” *Sci. Rep.*, vol. 2, pp. 725–733, 2012.
- [37] M. Birattari, “On the estimation of the expected performance of a metaheuristic on a class of instances. how many instances, how many runs?” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2004-001, 2004.



**Huw Lloyd** is a Senior Lecturer at Manchester Metropolitan University. He was awarded his B.Sc. in Physics by Imperial College, London, and his Ph.D. in Astrophysics by the University of Manchester.



**Martyn Amos** is Professor of Computer and Information Sciences at Northumbria University. He was awarded his B.Sc. in Computer Science by Coventry University, and his Ph.D. in DNA computation by the University of Warwick. He is a Fellow of the British Computer Society.